# AIAC ASSIGNMENT – 16.2

**Name : D.Akash**

**H.No : 2303A51422**

**Batch : 21**

**Task Descrption-1**: (Design Database Schema for Student Management System)

Instructions:

• Using AI tools, generate CREATE TABLE statements for a Student Management Database.

• Tables: Students, Courses, Enrollments.

• Include primary keys, foreign keys, and suitable data types.

Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

---

**Code :**

AIAC-16.2.sql ● {} [{"stmt":"CREATE TABLE Students (\n stud Untitled-3 ● <> //utils.js Untitled-2 ● JS const express =

AIAC-16.2.sql

```sql
-- SQLite
--Task Descrption-1: (Design Database Schema for Student Management System)
--Instructions:
-- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
-- Tables: Students, Courses, Enrollments.
-- Include primary keys, foreign keys, and suitable data types.
--Ensure the schema follows normalization rules (1NF, 2NF, 3NF).
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    date_of_birth DATE NOT NULL
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);
```

---

---

## Output :

SQLite

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
date_of_birth DATE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
course_code TEXT UNIQUE NOT NULL,
credits INTEGER NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
enrollment_date DATE NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id),
UNIQUE(student_id, course_id)
);
```

---

**Observation :**

The AI assistant makes creating database tables much easier because it generates clear and structured SQL code automatically. Instead of writing everything from scratch, the AI provides ready-to-use CREATE TABLE statements with primary keys, foreign keys, and constraints already included. This saves time, reduces errors, and shows how AI can simplify complex tasks like designing a normalized database schema.

**Task Description- 2**: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the

tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

**Code :**

```sql
/*Task Description- 2: (Insert Sample Records)
Instructions:
• Use AI tools to generate INSERT statements to populate the
tables.
• Insert at least 3 students, 4 courses, and 5 enrollment records.
Verify data using SELECT queries.
*/
INSERT INTO Students (first_name, last_name, email, date_of_birth) VALUES
('John', 'Doe', 'john.doe@example.com', '2000-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2001-05-15'),
('Emily', 'Johnson', 'emily.johnson@example.com', '2002-08-20');
INSERT INTO Courses (course_name, course_code, credits) VALUES
('Introduction to Computer Science', 'CS101', 3),
('Data Structures', 'CS102', 4),
('Database Systems', 'CS103', 3),
('Operating Systems', 'CS104', 4);
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
(1, 1, '2023-09-01'),
(1, 2, '2023-09-01'),
(2, 1, '2023-09-01'),
(2, 3, '2023-09-01'),
(3, 4, '2023-09-01');
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

```sql
INSERT INTO Students (first_name, last_name, email, date_of_birth)
VALUES
('John', 'Doe', 'john.doe@example.com', '2000-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2001-05-15'),
('Emily', 'Johnson', 'emily.johnson@example.com', '2002-08-20');
```

```sql
INSERT INTO Courses (course_name, course_code, credits) VALUES
('Introduction to Computer Science', 'CS101', 3),
('Data Structures', 'CS102', 4),
('Database Systems', 'CS103', 3),
('Operating Systems', 'CS104', 4);
```

```sql
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
(1, 1, '2023-09-01'),
(1, 2, '2023-09-01'),
(2, 1, '2023-09-01'),
(2, 3, '2023-09-01'),
(3, 4, '2023-09-01');
```

**Output :**

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT * FROM Students;
```

| student_id | first_name | last_name | email | date_of_birth |
|---|---|---|---|---|
| 1 | John | Doe | john.doe@example.com | 2000-01-01 |
| 2 | Jane | Smith | jane.smith@example.com | 2001-05-15 |
| 3 | Emily | Johnson | emily.johnson@example.com | 2002-08-20 |

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

| course_id | course_name | course_code | credits |
|---|---|---|---|
| 1 | Introduction to Computer Science | CS101 | 3 |
| 2 | Data Structures | CS102 | 4 |
| 3 | Database Systems | CS103 | 3 |
| 4 | Operating Systems | CS104 | 4 |

SQL ▾ 1 / 1 1 - 5 of 5

```
SELECT * FROM Enrollments;
```

| enrollment_id | student_id | course_id | enrollment_date |
|---|---|---|---|
| 1 | 1 | 1 | 2023-09-01 |
| 2 | 1 | 2 | 2023-09-01 |
| 3 | 2 | 1 | 2023-09-01 |
| 4 | 2 | 3 | 2023-09-01 |
| 5 | 3 | 4 | 2023-09-01 |

**Observation :**

The AI assistant makes inserting sample records very straightforward. Instead of manually thinking up values and writing multiple INSERT statements, the AI generates complete, ready-to-use SQL code with realistic student names, course details, and enrollment records. This not only saves time but also reduces the chance of mistakes, while giving clear examples that make it easy to understand how the database works in practice.

**Task Description- 3**: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student’s email
- Delete an enrollment record
- Validate the changes using SELECT statements.

**Code :**

```sql
/*Task Description- 3: (Perform CRUD Operations)
Instructions:
Use AI to generate SQL queries for basic CRUD operations:
• Retrieve all courses
• Update a student’s email
• Delete an enrollment record
• Validate the changes using SELECT statements.
*/
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student’s email
UPDATE Students SET email = 'john.newemail@example.com' WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments WHERE enrollment_id = 1;
-- Validate the changes using SELECT statements
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

**Output :**

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

| course_id | course_name | course_code | credits |
|---|---|---|---|
| 1 | Introduction to Computer Science | CS101 | 3 |
| 2 | Data Structures | CS102 | 4 |
| 3 | Database Systems | CS103 | 3 |
| 4 | Operating Systems | CS104 | 4 |

SQL ▾ 1 / 1 1 - 0 of 0

```
UPDATE Students SET email = 'john.newemail@example.com' WHERE student_id
= 1;
```

SQL ▾ 1 / 1 1 - 0 of 0

```
DELETE FROM Enrollments WHERE enrollment_id = 1;
```

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT * FROM Students;
```

| student_id | first_name | last_name | email | date_of_birth |
|---|---|---|---|---|
| 1 | John | Doe | john.newemail@example.com | 2000-01-01 |
| 2 | Jane | Smith | jane.smith@example.com | 2001-05-15 |
| 3 | Emily | Johnson | emily.johnson@example.com | 2002-08-20 |

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Enrollments;
```

| enrollment_id | student_id | course_id | enrollment_date |
|---|---|---|---|
| 2 | 1 | 2 | 2023-09-01 |
| 3 | 2 | 1 | 2023-09-01 |
| 4 | 2 | 3 | 2023-09-01 |
| 5 | 3 | 4 | 2023-09-01 |

**Observation :**

The AI assistant makes performing CRUD operations very easy by generating clear SQL queries for each task. Instead of figuring out the exact syntax manually, the AI provides ready-made statements to retrieve data, update records, and delete entries, along with validation queries to check the changes. This helps save time, reduces errors, and shows how AI can simplify database management tasks in a practical way.

**Task Description- 4**: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

• Count number of students enrolled per course

• Display courses with more than one student enrolled

Analyze the correctness of results.

**Code :**

```
/*Task Description- 4: (Execute Aggregate Queries)
Instructions:
Use AI tools to write aggregate queries:
• Count number of students enrolled per course
• Display courses with more than one student enrolled
Analyze the correctness of results.
*/

-- Count number of students enrolled per course
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
-- Display courses with more than one student enrolled
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING student_count > 1;

--explain why the result is empty for the second query
/*The result is empty for the second query because there are no courses that have more than one
student enrolled. Each course in the Enrollments table has only one student enrolled, so the
HAVING  clause filters out all courses, resulting in an empty result set.*/
```

**Output :**

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
```

| course_id | student_count |
|---|---|
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |

SQL ▾ 1 / 1 1 - 0 of 0

```
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING student_count > 1;
```

**Observation :**

The AI assistant makes writing aggregate queries simple by automatically generating the correct SQL syntax for tasks like

counting students per course and filtering courses with more than one student. It also explains the results clearly—for example, why the second query returned empty—showing that the AI doesn't just give code but also helps in understanding the logic behind it. This makes learning and verifying aggregate functions much easier and more intuitive.

**Task Description- 5**: (Query Optimization)

Instructions:

• Use AI tools to analyze query performance.

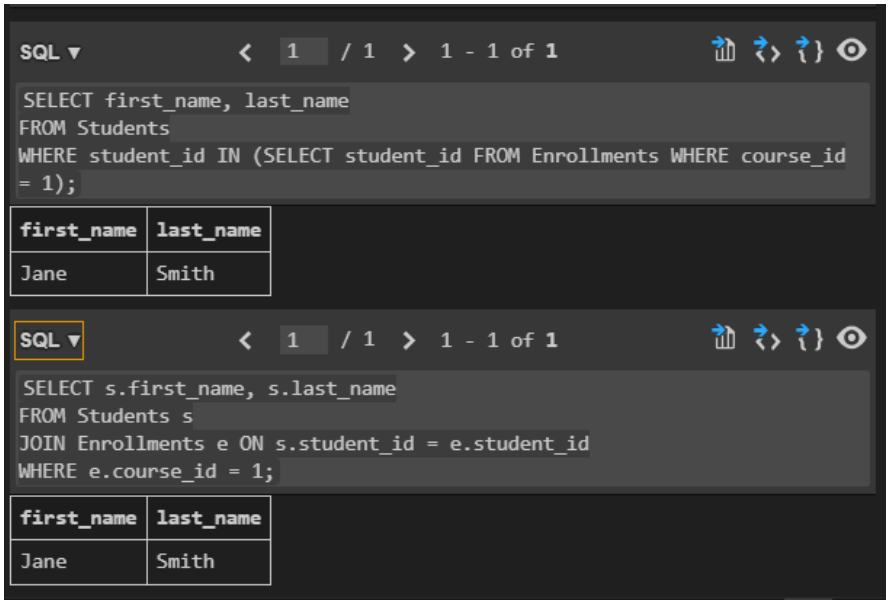
• Rewrite inefficient sub queries using JOINs and test both queries

for correctness.

**Code :**

```
/*Task Description- 5: (Query Optimization)
Instructions:
• Use AI tools to analyze query performance.
• Rewrite inefficient sub queries using JOINs and test both queries
for correctness.*/
-- Inefficient subquery to retrieve student names enrolled in a specific course
SELECT first_name, last_name
FROM Students
WHERE student_id IN (SELECT student_id FROM Enrollments WHERE course_id = 1);
-- Optimized query using JOINs
SELECT s.first_name, s.last_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

**Output :**




**Observation :**

The AI assistant helps in improving query performance by analyzing the code and suggesting optimized versions. For example, it rewrites an inefficient subquery into a JOIN, which makes the query faster and easier to understand. This shows that the AI doesn't just generate working SQL, but also guides toward better practices, making database operations more efficient and practical to use in real scenarios.